



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Катарина Вељковић

Развој и мрежна синхронизација игре „Потапање подморница“ уз примену криптографске заштите

ДИПЛОМСКИ РАД
- Основне академске студије -

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:		
Идентификациони број, ИБР:		
Тип документације, ТД:	Монографска документација	
Тип записа, ТЗ:	Текстуални штампани материјал	
Врста рада, ВР:	Завршни-bachelor рад	
Аутор, АУ:	Катарина Вељковић	
Ментор, МН:	Ванр. проф. др Милана Бојанић	
Наслов рада, НР:	Развој и мрежна синхронизација игре „Потапање подморница“ уз примену криптографске заштите	
Језик публикације, ЈП:	Српски (ћирилица)	
Језик извода, ЈИ:	Српски	
Земља публиковања, ЗП:	Република Србија	
Уже географско подручје, УГП:	Војводина	
Година, ГО:	2026	
Издавач, ИЗ:	Ауторски репринт	
Место и адреса, МА:	Нови Сад; трг Доситеја Обрадовића 6	
Физички опис рада, ФО: (поглавља/страна/цитата/табела/слика/графика/прилога)	7/30/0/1/21/0/0	
Научна област, НО:	Електротехника и рачунарство	
Научна дисциплина, НД:	Примењено софтверско инжењерство	
Предметна одредница/Кључне речи, ПО:	Мрежна игра, серијализација, C#, WPF, TCP/UDP, AES-256	
УДК		
Чува се, ЧУ:	Библиотека ФТН, Д. Обрадовића 6, 21000 Нови Сад	
Важна напомена, ВН:		
Извод, ИЗ:	Овај дипломски рад описује развој неблокирајућег клијент-сервер система за мрежну игру „Потапање подморница“ коришћењем C# програмског језика и WPF оквира. Систем користи хибридни мрежни приступ: UDP протокол за брзу регистрацију играча и TCP протокол за сигурну размену података током партије, уз примену AES-256 криптографске заштите порука. Рад детаљно приказује архитектуру система, укључујући централизовану логику на серверу, валидацију правила игре, механизам "супер-потеза" (3x3), као и систем аутоматског преузимања контроле (бот) у случају неактивности играча, чиме се обезбеђује континуитет и поузданост игре у реалном времену.	
Датум прихватања теме, ДП:		
Датум одбране, ДО:		
Чланови комисије, КО:	Председник:	
	Члан:	Потпис ментора
	Члан, ментор:	Др Милана Бојанић, ванр. проф.



KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	Bachelor thesis
Author, AU :	Katarina Veljković
Mentor, MN :	Milana Bojanić, PhD, Associate professor
Title, TI :	Development and network synchronization of the 'Battleship' game with cryptographic protection
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/30/0/1/21/0/0
Scientific field, SF :	Electrical Engineering
Scientific discipline, SD :	Applied software engineering
Subject/Key words, S/KW :	Network game, Serialization, C#, WPF, TCP/UDP, AES-256
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad, Serbia
Note, N :	
Abstract, AB :	This bachelor thesis describes the development of a non-blocking client-server system for the network game "Battleship" using the C# programming language and the WPF framework. The system employs a hybrid networking approach: UDP protocol for fast player registration and TCP protocol for secure data exchange during the match, incorporating AES-256 cryptographic protection for messages. The thesis details the system architecture, including centralized server-side logic, game rule validation, a "super-move" mechanism (3x3), as well as an automatic control takeover system (BOT) in case of player inactivity, ensuring real-time game continuity and reliability.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	
Defended Board, DB :	President:
	Member:
	Member, Mentor:
	Menthor's sign
	Milana Bojanić, PhD, Associate Professor

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Датум:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Лист/Листова:
		1/1

(Податке уноси предметни наставник - ментор)

Врста студија:	☒ Основне академске студије
Студијски програм:	Примењено софтверско инжењерство
Руководилац студијског програма:	др Александар Селаков

Студент:	Катарина Вељковић	Број индекса:	PR 33/2022
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Ванр. проф. др Милана Бојанић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: 1. проблем – тема рада; 2. начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна; 3. литература			

НАСЛОВ ЗАВРШНОГ РАДА:

РАЗВОЈ И МРЕЖНА СИНХРОНИЗАЦИЈА ИГРЕ „ПОТАПАЊЕ ПОДМОРНИЦА“ УЗ ПРИМЕНУ КРИПТОГРАФСКЕ ЗАШТИТЕ

ТЕКСТ ЗАДАТКА:

Задатак завршног рада је дизајнирање и имплементација клијент-сервер апликације за игру „Потапање подморница“, која подржава истовремено играње више парова играча у дистрибуираном окружењу. Систем мора да обезбеди стабилну мрежну комуникацију, сигурност података и континуитет игре кроз аутоматизоване процесе. Применити клијент-сервер модел са централизованом логиком на серверу за валидацију правила игре. За иницијалну регистрацију играча и брзу размену статуса употребити UDP протокол, а за поуздану размену података током партије и управљање потезима применити TCP протокол. Ради безбедности комуникације применити AES-256 криптографску заштиту над свим порукама које се размењују између клијента и сервера како би се спречило неовлашћено пресретање. Поред имплементације основне логике игре, подржати и допунски механизам „супер-потеза“ (дејство по пољу 3x3), као и примену аутоматизованог система (бот) који преузима контролу над игром у случају да играч постане неактиван.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА

21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1. Увод.....	3
2. Теоријске основе рада	4
2.1 Мрежни протоколи TCP и UDP	4
2.1.1 UDP (<i>User Datagram Protocol</i>)	4
2.1.2 TCP (<i>Transmission Control Protocol</i>)	4
2.1.3 Сличности и разлике између TCP и UDP	5
2.2 Криптографска заштита порука: AES-256	5
2.2.1 Принцип рада AES алгоритма	5
2.2.2 Предности и мане AES алгоритма	6
3. Опис коришћених технологија и алата	7
3.1. Технологије серверске стране.....	7
3.2. Криптографска заштита и апликативни протокол	7
3.3. Алати и окружење	8
4. Мрежна игра „Потапање подморница“	9
4.1 Дијаграм случаја употребе игре	9
4.1.1 Актери система.....	10
4.1.2 Случајеви употребе.....	10
4.2 Дизајн система мрежне игре.....	11
4.2.1 Компонентни дизајн.....	11
4.2.2 Комуникациони дизајн.....	12
4.2.3 Секвенца кључних сценарија	12
4.2.4 Дизајн одговорности и контрола стања.....	13
4.2.5 Дизајнерске предности изабраног решења	13
5. Имплементација игре.....	14
5.1 Архитектура решења.....	14
5.2 Структура података и заједнички модели.....	14
5.2.1 Класа <i>Podmornica</i> и структурирање флоте	14
5.2.2 Класа <i>Igrac</i> и управљање меморијским стањем	14
5.2.3 Класа <i>Poruka</i> као апликативни мрежни протокол	15
5.3 Клијентска апликација (WPF специфичности)	17
5.4 Логика иницијализације и регистрације играча	17
5.5 Валидација и правила постављања подморница.....	18
5.6 Логика играња потеза.....	19
5.7 Специјални супер потез (9 поља одједном).....	20
5.8 Временско ограничење, казнени поени и бот преузимање контроле	20
5.9 Бележење тока игре у документ.....	22

6. Верификација квалитета решења.....	23
6.1 Покретање и повезивање	23
6.2 Постављање подморница и припрема игре.....	24
6.3 Ток игре и комуникација	25
6.4 Механизам аутоматизације (играчки бот)	26
6.5 Крај игре и нова игра	26
7. Закључак	28
7.1 Унапређење постојећег софтверског решења.....	28
Литература	29
Подаци о кандидату	30

1. Увод

Развој савремених мрежних софтверских система захтева не само поуздану размену података, већ и механизме који омогућавају брзу синхронизацију и истовремену интеракцију више корисника у реалном времену. У домену дистрибуираних апликација, развој вишекорисничких система представља комплексан инжењерски изазов где је транспарентност и конзистентност стања на мрежи од кључног значаја.

Анализирајући архитектуру мрежних игара, посебно оних које почивају на конкурентним, потезним (енгл. *turn-based*) принципима — на пример рачунарски шах — јасно се издваја неколико критичних слојева проблема.

Код оваквих система, поред чисте имплементације правила игре и корисничког интерфејса, од суштинског је значаја успоставити правилну инфраструктуру за комуникацију. Комуникациони изазови у мрежним играма обухватају проблеме као што су латенција (кашњење), губитак пакета у саобраћају, несигурне мрежне путање и несинхронизовани сатови између клијената и сервера.

Традиционални приступи мрежној комуникацији засновани искључиво на једном протоколу носе са собом низ недостатака: стварају велико оптерећење за хардверске ресурсе, пате од честих блокирања рада система и доводе до губитка кључних података на несигурним мрежним линијама. Корисници у таквим системима често доживљавају десинхронизацију стања игре, што онемогућава правилно праћење тока партије и нарушава корисничко искуство.

Због тога је приликом пројектовања неопходно пажљиво размотрити избор комуникационих протокола. Док протоколи базирани на корисничким датаграмима омогућавају брзи пренос података уз минимално време чекања, протоколи оријентисани ка конекцији гарантују тачност и редослед испоруке пакета. Имплементација хибридних мрежних протокола као дела софтверске архитектуре представља решење које истовремено обезбеђује ниску латенцију и сигурност испоруке пакета [1]. На пример, фаза иницијализације, аутентификације и успостављања сесије захтева апсолутну поузданост и конзистентност, док се поједини сегменти преноса актуелних информација могу оптимизовати ради бржег одговора система.

Поред саме комуникације, правилна синхронизација података представља један од најтежих задатака. Синхронизација обухвата сигурно повезивање два играча у лобију, прецизну размену информација о потезима и перцепцији стања, као и одржавање идентичног, конзистентног стања игре код оба учесника у сваком тренутку. Сваки пропуст у редоследу обраде догађаја може довести до стања у којем један играч види једну ситуацију на табли, док други има сасвим другачију перцепцију истог догађаја, што руши регуларност игре. Задаци имплементације логике игре стога морају бити модуларно раздвојени од корисничког интерфејса, користећи савремене обрасце пројектовања како би се омогућило лако тестирање, одржавање и евентуално ширење функционалности система.

Циљ овог рада је развој комплетног система за мрежну игру „Потапање подморница” која демонстрира примену напредних концепата софтверског инжењерства. Систем подржава интерактивну игру за два играча, уз криптографску заштиту, ефикасну синхронизацију преко хибридних мрежних модела и добро корисничко искуство. Кроз поглавља која следе биће детаљно приказани анализа захтева, архитектонско решење, избор протокола и имплементациони детаљи целокупног решења.

2. Теоријске основе рада

Развој савремених мрежних софтверских система и интерактивних апликација захтева темељно разумевање основних комуникационих протокола и метода заштите података. У контексту архитектуре мрежне игре „Потапање подморница“, кључни стубови техничке реализације почивају на синергији различитих мрежних парадигми (TCP и UDP) и механизмима криптографске заштите (AES-256). Ово поглавље даје детаљан теоријски преглед поменутих технологија, њихових принципа рада, предности, недостатака и практичних примена.

2.1 Мрежни протоколи TCP и UDP

Два најзначајнија и доминантна протокола транспортног слоја су TCP (*Transmission Control Protocol*) и UDP (*User Datagram Protocol*) [2]. Иако оба користе Интернет протокол (IP) за усмеравање пакета ка одредишту, њихов филозофски и функционални приступ преносу података суштински се разликује.

2.1.1 UDP (*User Datagram Protocol*)

UDP је протокол без успоставе везе (енгл. *connectionless*) оријентисан ка слању датаграма за које не гарантује поуздану испоруку. То значи да пре успостављања саме размене података не постоји фаза „руковања“, тј. успоставе везе (енгл. *handshake*) између пошиљаоца и примаоца, већ пошиљалац једноставно шаље пакет (датаграм) ка одредишту без претходне провере да ли је прималац спреман или доступан.

- **Карактеристике и принципи комуникације:** UDP не гарантује испоруку пакета, не прати редослед којим пакети стижу и не поседује уграђене механизме за ретрансмисију изгубљених података. Сваки датаграм је потпуно независна јединица. Због одсуства контролних механизма и чекања потврда (ACK) обезбеђује изузетно малу латенцију.
- **Најчешће примене:** UDP се користи у сценаријима где је брзина преноса критичнија од апсолутне тачности сваког појединог пакета. Примери укључују стримовање видео и аудио садржаја, DNS упите, онлајн игре у реалном времену (за праћење брзих позиција играча) и апликације за брзу почетну детекцију (као што је регистрација и откривање активних играча у лобију мрежне игре „Потапање подморница“).

2.1.2 TCP (*Transmission Control Protocol*)

За разлику од UDP-а, TCP је конекционо оријентисан (енгл. *connection-oriented*), поуздан протокол који обезбеђује испоруку података без губитака и у тачно утврђеном редоследу.

- **Карактеристике и принципи комуникације:** Комуникација путем TCP-а почиње троструким руковањем (енгл. *three-way handshake*) — разменом SYN, SYN-ACK и ACK пакета између клијента и сервера ради синхронизације почетних секвенци бројева и успостављања везе. Током трајања сесије, TCP користи механизме потврђивања пријема пакета (ACK бројеви потврде), контролу протока како би спречио преоптерећење примаоца, као и контролу загушења мреже. Уколико дође до губитка пакета на мрежи, протокол аутоматски захтева и врши ретрансмисију изгубљеног садржаја.
- **Најчешће примене:** TCP је стандард за све апликације код којих је интегритет података од највеће важности. То обухвата пренос садржаја веб сајтова

(HTTP/HTTPS), пренос фајлова (FTP), слање е-поште (SMTP) и размену кључних потеза у потезним мрежним играма (као што је слање гађања и промена табли у игри „Потапање подморница“, где губитак или погрешан редослед једног потеза потпуно руши регуларност партије).

2.1.3 Сличности и разлике између TCP и UDP

Димензија поређења	TCP	UDP
Тип везе	Конекционо оријентисан (захтева успостављање везе)	Без конекције (датаграмски систем)
Поузданост	Висока (гарантује испоруку и редослед)	Ниска (не гарантује испоруку, нити редослед)
Брзина / Латенција	Спорији због ревизија, потврда и контроле загушења	Изузетно брз због минималног заглавља и одсуства потврда
Контрола протока и грешака	Постоји (детекција грешака, ретрансмисија)	Не постоји (обавеза апликативног слоја, ако је потребна)
Величина заглавља	Већа (минимум 20 бајтова)	Мала (фиксних 8 бајтова)

Табела 2.1. Приказ сличности и разлика између TCP и UDP протокола

2.2 Криптографска заштита порука: AES-256

У савременим дистрибуираним системима и мрежним играма, заштита комуникационих канала од неовлашћеног прислушкивања, пресретања и злонамерне модификације података представља најбитнију тачку. У ту сврху се користе напредни криптографски стандарди, међу којима апсолутну доминацију има AES (*Advanced Encryption Standard*) [3].

2.2.1 Принцип рада AES алгоритма

AES је симетрични блоковски шифарски алгоритам, што значи да се исти тајни кључ користи и за процесе шифровања (претварања читљивог текста у шифрат) и за процесе дешифровања (враћања шифрата у првобитни читљив облик).

Алгоритми из AES породице оперативно раде над фиксним блоковима података величине 128 битова, док дужина тајног кључа може варирати (128, 192 или 256 битова). Варијанта AES-256 користи кључ дужине 256 битова, што јој пружа веома висок ниво сигурности.

Структура AES алгоритма заснована је на такозваној *Substitution-Permutation Network* (SPN) архитектури и састоји се од више итеративних рунди трансформација (за 256-битни кључ извршава се укупно 14 рунди). Свака стандардна рунда обухвата четири основне трансформације над блоком података (бајтовима блока):

-
1. *SubBytes* (Замена бајтова): Нелинеарна замена сваког бајта независно, коришћењем пређених вредности из фиксне S-кутије (енгл. *Substitution Box*), што обезбеђује криптографску конфузију.
 2. *ShiftRows* (Померање редова): Циклично померање бајтова у редовима матрице за одређени број места, чиме се постиже ширење утицаја појединачних бајтова по целом блоку (дифузија).
 3. *MixColumns* (Мешање колона): Операција мешања података у оквиру сваке колоне појединачно помоћу матричног множења у Галоовом пољу, што додатно ојачава дифузију.
 4. *AddRoundKey* (Додавање кључа): Кључ специфичан за текућу рунду (изведен из главног кључа кроз процес ширења кључа — Key Expansion) комбинује се са матрицом података помоћу логичке операције XOR.
- **Процес шифровања и дешифровања:** Пошиљалац узима читљиву мрежну поруку (на пример, координате гађања у игри), дели је на блокове, примењује скуп горенаведених трансформација помоћу тајног кључа и генерише нечитљиви низ бајтова (шифрат) који се шаље преко TCP мреже. Прималац по пријему шифрата, користећи исти тајни кључ, примењује реверзибилне инверзне операције у обрнутом редоследу, како би успешно реконструисао оригиналну поруку.
 - **Управљање кључевима:** Будући да је реч о симетричној криптографији, кључна претпоставка безбедности је сигурна размена и складиштење тајног кључа између клијента и сервера приликом иницијализације сесије, или његово извођење кроз безбедне протоколе за размену кључева.

2.2.2 Предности и мане AES алгоритма

- **Предности:**
 - Висок ниво сигурности: AES са кључем дужине 256 бита пружа веома висок ниво криптографске сигурности и сматра се отпорним на нападе грубом силом уз тренутно доступну рачунарску технологију.
 - Хардверска оптимизација: Велики број савремених процесора подржава хардверску реализацију AES операција, као што су AES-NI инструкције, што омогућава брзо шифровање и дешифровање уз минимално оптерећење процесора.
 - Стандардизација и широка примена: AES је стандардизован од стране америчког Националног института за стандарде и технологију (NIST) и подржан је у великом броју система, протокола и апликација.
- **Мане:**
 - Проблем дистрибуције кључева: Као и код свих симетричних криптографских алгоритама, један од основних проблема представља безбедна размена тајног кључа између комуникационих страна пре успостављања заштићене комуникације.
 - Фиксна величина блока: AES користи блокове фиксне величине од 128 бита, што може захтевати додатно руковање подацима у зависности од изабраног режима рада. Неадекватна имплементација режима рада и руковања допуњавањем може довести до безбедносних пропуста, укључујући нападе типа *padding oracle*.

3. Опис коришћених технологија и алата

Избор технологија, алата и сигурностних механизма за развој овог система заснован је на специфичним захтевима пројекта, при чему је приоритет био постизање неблокирајућег рада сервера, поузданости и сигурности преноса података у реалном времену, као и ефикасно управљање стањима игре.

Клијент-сервер архитектура захтева технологије које омогућавају сигурно управљање мрежним утичницама, истовремену обраду конкурентних захтева, заштиту поверљивости саобраћаја и поуздану серијализацију дељених структура података. У наставку су описане кључне технологије и криптографски механизми коришћени за имплементацију серверске и клијентске стране система.

3.1. Технологије серверске стране

.NET платформа и **C#** су изабрани као основа за развој серверске стране система због своје стабилности, високих перформанси и напредне подршке за нискостепено мрежно програмирање. Архитектура вођена догађајима и асинхрони рад унутар .NET окружења омогућавају ефикасну симулацију потезног система [4]. Програмски језик C# нуди богату палету уграђених библиотека за манипулацију колекцијама података, токовима меморије и криптографским трансформацијама, што убрзава развој сложене логике као што је управљање табелама играча, рачунање погодака и координација аутоматизованих ботова.

System.Net.Sockets простор имена и мрежне утичнице коришћени су за реализацију комплетног мрежног комуникационог слоја. На серверској страни примењен је неблокирајући режим рада, који омогућава серверу да прихвата захтеве играча и процесира поруке без заустављања извршне нити. Овај приступ је кључан за очување перформанси система у тренуцима када се чека унос од стране клијената.

Socket.Select механизам је искоришћен као метод мултиплексирања мрежних утичница на серверу. Уместо креирања посебних извршних нити за сваког повезаног играча – што би резултирало великим хардверским оптерећењем – функција **Socket.Select** омогућава серверу да унутар једне јединствене нити надгледа комплетну листу активних клијената [5]. Систем блокира извршавање само када на мрежи нема никаквих дешавања, а чим било која утичница прими пакет, функција га филтрира као спремног за читање, омогућавајући серверу да брзо и серијски обради пристигле потезе.

UDP је изабран за почетну фазу регистрације и откривања играча због свог бесконекционог карактера и минималног кашњења у испоруци пакета, ослањајући се на класичне теоријске поставке транспортних слојева мреже. Клијенти шаљу почетне информације за пријаву играча у виду UDP датаграма, што омогућава брзу детекцију присутних корисника.

TCP је коришћен за комплетан даљи ток игре. С обзиром на то да логика игре не дозвољава губитак пакета нити промену њиховог редоследа, TCP кроз своје механизме контроле протока и потврде пријема гарантује да ће сваки одиграни потез и промена стања табле бити поуздано испоручени.

3.2. Криптографска заштита и апликативни протокол

Ради испуњавања захтева за безбедност и поверљивост података који се размјењују између клијената и сервера, у мрежни слој је интегрисана снажна криптографска заштита.

AES-256 (Advanced Encryption Standard са кључем дужине 256 бита) примењен је као стандард за симетрично шифровање целокупног мрежног саобраћаја након успостављања TCP везе. Сви осетљиви подаци, укључујући команде за гађање, координате подморница и синхронизацију стања игре, шифрују се пре слања кроз мрежу, чиме се онемогућава прислушкивање или злонамерна измена пакета од стране трећих лица.

Класа Toruka и дељени модели чине апликативни протокол система који омогућава структурирану и заштићену размену података између клијента и сервера. Кроз енумерацију TipToruke, сваком шифрованом пакету се придружује јасан контекст (као што су Napad, Pogodak, Promasaj, TajmerOdbrojavanje), што омогућава обема странама да на дешифрованој страни моменат покрену одговарајућу логику за ажурирање приказа или стања у меморији.

Бинарна серијализација је примењена за претварање комплексних објеката дељених класа у низ бајтова погодан за шифровање и слање кроз мрежне стримове. Коришћењем класе BinaryFormatter и тока меморије MemoryStream, комплетни објекти који садрже податке о играчу, нападачу и порукама се пре процеса енкрипције аутоматски серијализују, док се на пријемној страни врши обрнути процес дешифровања и десеријализације [6]. Овај приступ елиминисао је потребу за ручним парсирањем текстуалних стрингова и гарантује типску безбедност података на мрежи.

System.Timers.Timer је искоришћен како би се у игру увео динамички елемент у виду временског ограничења за одигравање потеза. Тајмер ради у засебној нити и одбројава унапред дефинисано време за унос потеза. Уколико време истекне, тајмер окида догађај који аутоматски пребацује контролу на BotHelper класу која симулира потез рачунарског играча, обезбеђујући континуитет игре без застоја.

3.3. Алати и окружење

Microsoft Visual Studio је коришћен као главно интегрисано развојно окружење због своје изузетне подршке за C# и .NET апликације. Интегрисани напредни алати за проналажење грешака (енгл. *debugger*) омогућили су симултано покретање и праћење рада једне инстанце сервера и више инстанци конзолних клијената истовремено, што је било од кључног значаја за праћење рада криптографских бафера и мрежних токова у реалном времену.

System.IO.StreamWriter и систем логовања искориштени су за имплементацију централизоване класе Logger. Овај алат омогућава аутоматско генерисање текстуалних датотека унутар "Logs" директоријума за сваку нову партију [7]. Кроз увођење механизма закључавања, обезбеђен је сигуран упис података из различитих нити, чиме се постиже поуздана перзистенција и могућност касније ревизије и анализе свих догађаја, безбедносних параметара и грешака у систему.

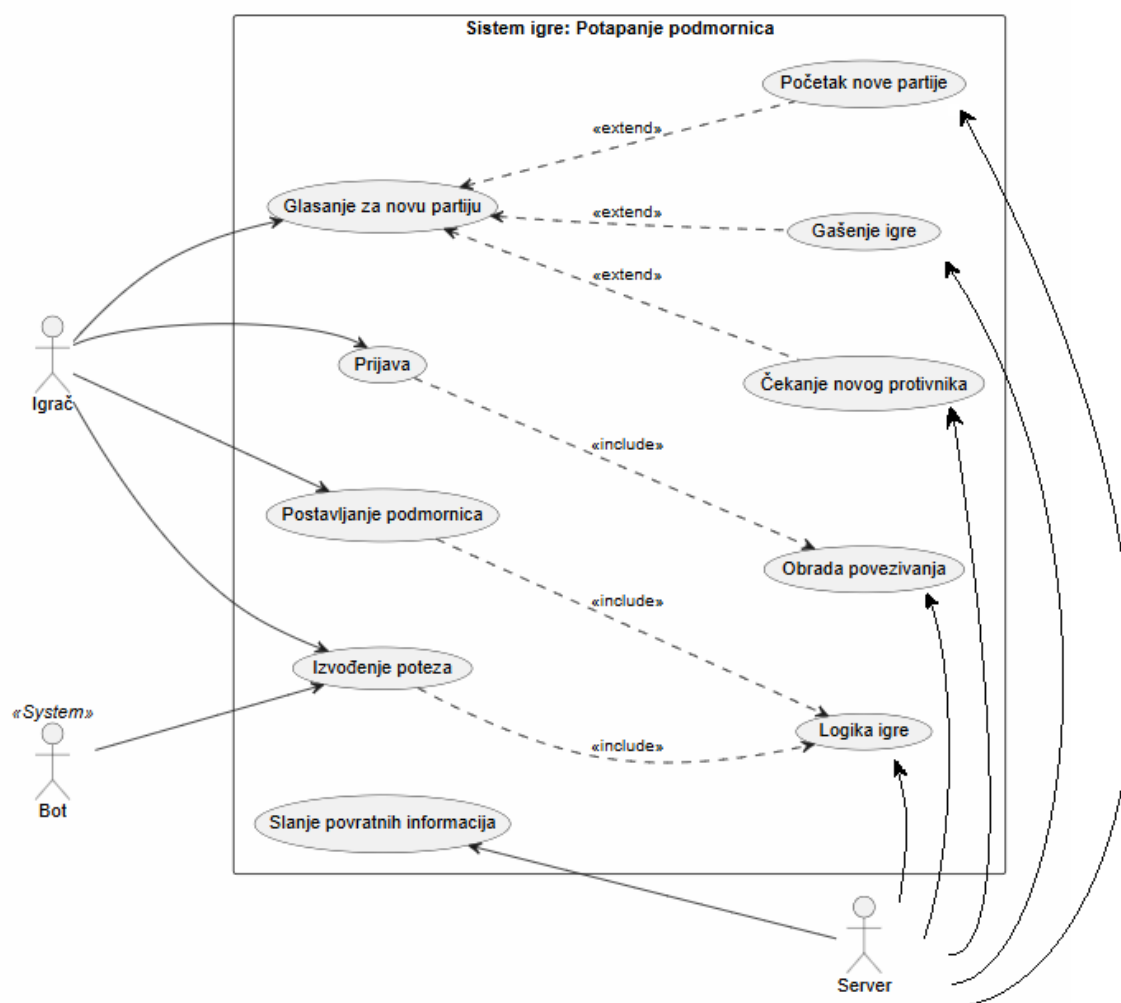
Git је коришћен за контролу верзија изворног кода, док је **GitHub** платформа употребљена за хостовање. Ово је омогућило паралелан развој клијентских функционалности и сервера, вођење чисте историје измена кроз комитове, као и сигурно спајање делова кода без ризика од конфликта.

4. Мрежна игра „Потапање подморница“

Ово поглавље описује реализацију мрежне игре „Потапање подморница“. Основна игра се одвија између играча којима је циљ да један играч потопи све подморнице на табли свог противника погађањем поља на којима се оне могу налазити. Комуникација између играча и централног сервера решена је комбинацијом UDP (User Datagram Protocol) протокола, који се користи за брзу почетну детекцију и регистрацију играча, и TCP (Transmission Control Protocol) протокола, који гарантује поуздану размену потеза и ажурирање табела током партије, као и криптографску заштиту пакета путем AES-256 алгорита.

Поред основних функционалности игре, имплементиране су и додатне могућности које доприносе комплексности и динамици система. Омогућено је постављање подморница различитих димензија путем WPF (*Windows Presentation Foundation* [8]) интерфејса, провера исправности њиховог позиционирања, као и употреба специјалног потеза који играч може искористити једном током партије. Додатно, уведено је временско ограничење за одигравање потеза, при чему систем аутоматски реагује уколико играч не одигра потез у предвиђеном времену. Такође је реализовано записивање тока игре у документ ради праћења активности играча и анализе рада система.

4.1 Дијаграм случаја употребе игре



Слика 4.1. Дијаграм случаја употребе система „Потапање подморница“

Дијаграм на слици 4.1. приказује комплетну интеракцију између актера и система. Систем је моделован као јединствена целина која управља логиком игре, а актери су играчи који интерактивно комуницирају са системом кроз различите операције.

4.1.1 Актери система

Играч - Реални корисник који се пријављује на систем преко WPF апликације, поставља подморнице, одиграва потезе и гласа за нову партију. Системски захтеви за играча су постављени у складу са официјалним правилима игре.

Бот - Аутоматизовани агент који делује као системска компонента. Бот се активира када играч прекорачи временско ограничење од 10 секунди. Његова функција је да спречи блокирање игре и гарантује континуитет партије. Он одиграва потез уместо пријављеног корисника.

Сервер - Централна системска компонента задужена за управљање стањем игре, координацију међу играчима, валидацију потеза, логовање и аутоматско управљање бот агентом.

4.1.2 Случајеви употребе

Пријава - Играч се пријављује на систем уносом јединственог имена. Систем валидира да назив није дуплиран и региструје играча. Ова операција је претпоставка за све остале операције у систему. Технички, пријава се обавља преко UDP протокола, што омогућава брзу детекцију играча на мрежи.

Обрада повезивања - Системска операција која процесира пријаву корисника на систем и повезује два пријављена играча у игру.

Постављање подморница - Припремна фаза где играч интерактивно позиционира 10 подморница различитих величина (1x1, 2x1, 3x1, 4x1) на 10x10 табли. Систем строго валидира геометријска правила: подморнице не смеју да се додирују (суседна поља морају бити празна), не смеју да излазе из граница табле и не смеју да се преклапају.

Извођење потеза - Главна динамичка фаза игре где су сви потези синхронизовани. Играч на потезу бира поље на противниковој табли, уз могућност једнократног коришћења супер потеза (9 поља одједном). Систем валидира да поље није већ гађано, проналази резултат (погодак, промашај, потапање) и ажурира стања табле. Сваки потез се преноси кроз TCP протокол, шифрован са AES-256 алгоритмом.

Логика игре - Системска операција која управља правилима игре: смењивањем потеза, временским ограничењима, казнама за узастопне промашаје, активација бота, обрадом резултата гађања и условима за крај партије.

Слање повратних информација - Систем континуирано шаље повратну информацију играчима: стање табле, резултате потеза, информације о стању игре и критичне поруке. Свака порука је шифрована AES-256 алгоритмом [\[3\]](#).

Гласање за нову партију - После завршене партије, оба играча гласају (Yes/No) да ли желе нову партију. Ако оба желе, партија се одмах почиње. Ако само један жели, систем чека 30 секунди за новог играча. Ако нов играч стигне, партија почиње; ако не, сервер се гаси.

Чекање новог противника - Играч чека 30 секунди на пријаву новог корисника, са којим би одиграо нову партију, након што је његов садашњи противник одустао од започињања нове игре.

Почетак нове партије - Уколико постоје услови за нову партију (оба играча су гласала "да" или нов играч је стигао), систем аутоматски реиницијализује табле и стање игре.

Гашење игре - Логички завршетак игре, након што су оба корисника гласала да не желе нову партију.

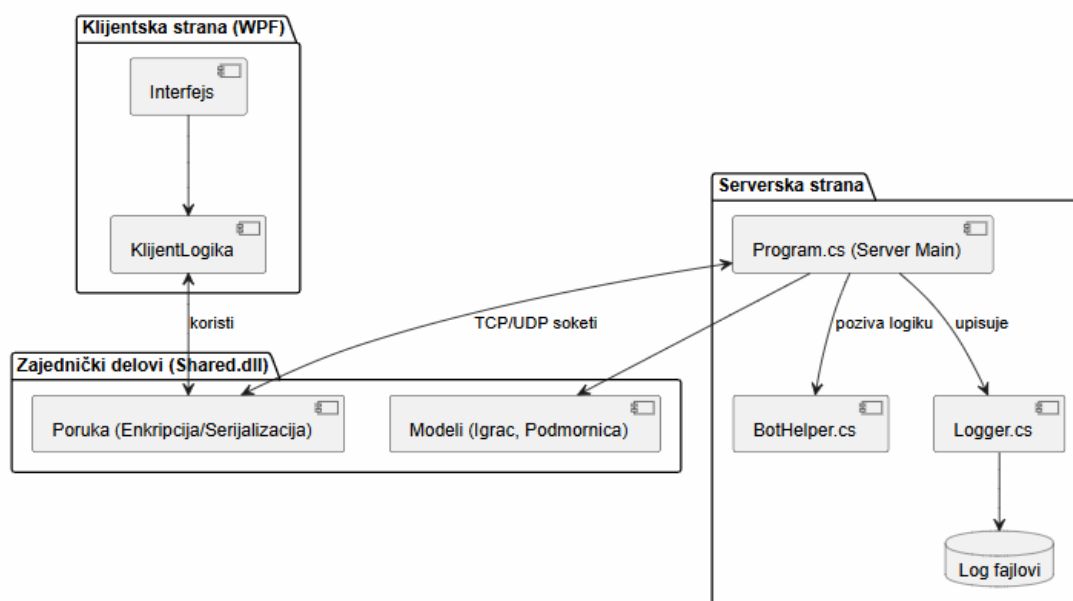
4.2 Дизајн система мрежне игре

Дизајн софтверског решења заснован је на дистрибуираној клијент–сервер архитектури у којој је WPF (*Windows Presentation Foundation*) клијент задужен за интеракцију са корисником, док сервер представља централни ауторитет над стањем игре и правилима. Заједнички слој (Shared.dll) садржи моделе и поруке које се користе на обе стране, чиме је обезбеђена типска усаглашеност комуникације.

Архитектура је пројектована тако да:

- кориснички интерфејс остане реактиван током целе партије,
- валидација кључних правила буде централизована на серверу,
- ток игре буде синхронизован за све учеснике,
- систем остане отпоран на неактивност играча (timeout + бот механизам).

4.2.1 Компонентни дизајн



Слика 4.2. Компонентни приказ система

Систем је подељен на три целине:

1. Клијентска страна

Састоји се од интерфејсног слоја и слоја клијентске логики.

- Интерфејс обезбеђује визуелизацију пријаве корисника, сопствене табле, противничке табле и свих потребних информација о току игре.
- Клијентска логика обрађује корисничке акције и комуникацију са сервером.

2. Заједнички слој

Садржи:

- класу Poruka (серијализација/шифровање/размена података),
- доменске моделе (Igrac, Podmornica и пратеће типове).

Овај слој дефинише „уговор“ комуникације између WPF клијента и сервера.

3. Серверска страна

Главни улаз је Program.cs, који оркестрира током партије:

- обраду мрежних захтева,
- позив доменске логике,
- активацију BotHelper у timeout сценаријима,
- логовање преко Logger класе у log фајлове.

4.2.2 Комуникациони дизајн

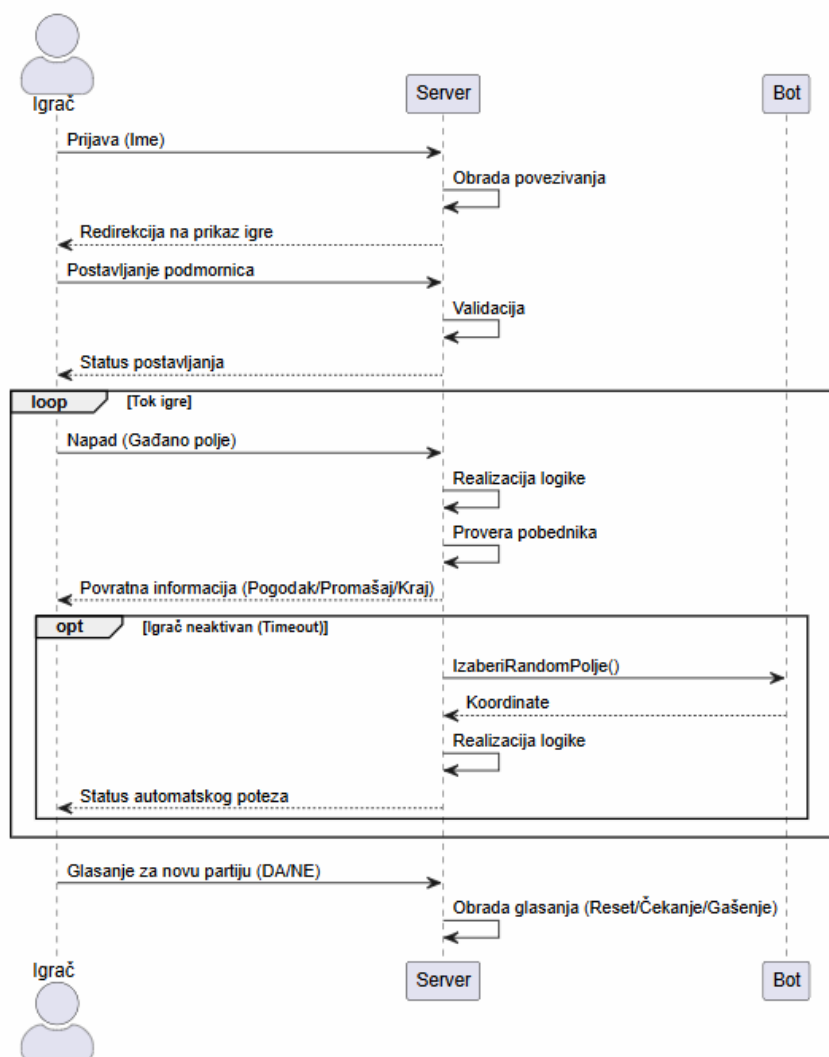
Комуникација је реализована употребом TCP/UDP сокета:

- **UDP** (*User Datagram Protocol*) се користи у иницијалној фази повезивања/регистрације [9]
- **TCP** (*Transmission Control Protocol*) се користи у главном току партије за поуздан пренос poteza и статуса [10]

Апликативни протокол је заснован на објекту поруке (Poruka), чиме се постиже:

- стандардизован формат размене података,
- једноставније шифровање,
- лакша обрада различитих типова догађаја,
- боља одрживост система у односу на ad-hoc текстуалне команде.

4.2.3 Секвенца кључних сценарија



4.3. Секвенцијални дијаграм

Секвенцијални дијаграм (Слика 4.3.) приказује комплетан ток сесије:

1. **Пријава играча** - Играч шаље име серверу; сервер обрађује повезивање и враћа статус.
2. **Поставка подморница** - Клијент шаље распоред својих подморница; сервер врши валидацију и враћа статус постављања.
3. **Ток игре** - Играч шаље напад (кликом изабраног поља на противничкој табли); сервер реализује логику, проверава услове победе и враћа резултат (погодак/промашај/крај).
4. **Timeout** - Ако је играч неактиван, сервер активира бота:
 - бот бира координате (`IzaberiRandomPolje()`),
 - сервер извршава аутоматски потез,
 - клијент добија статус аутоматског потеза.
5. **Гласање за нову партију** - По завршетку партије, сервер обрађује избор играча (ресет партије/чекање на новог противника/гашење игре).

4.2.4 Дизајн одговорности и контрола стања

Кључни принцип дизајна је централизована контрола стања на серверу: WPF клијент иницира акције, сервер потврђује валидност и ажурира стање, клијент приказује потврђени исход.

Оваква организација спречава десинхронизацију клијената и обезбеђује конзистентност партије. Бот механизам и логовање додатно унапређују поузданост система у условима неактивности или прекида корисничке интеракције.

4.2.5 Дизајнерске предности изабраног решења

Изабрани дизајн доноси следеће предности:

- јасна подела на презентациони, комуникациони и доменски део,
- лакше одржавање WPF интерфејса без измена серверске логики,
- централна валидација правила игре,
- отпорност на *timeout* сценарије,
- једноставнији процес шифровања порука,
- лакша анализа и ревизија партија путем логова.

5. Имплементација игре

5.1 Архитектура решења

Имплементација мрежне игре „Потапање подморница“ заснива се на вишеслојно организованом решењу које обухвата:

1. ClientWPF – графички кориснички интерфејс и клијентска логика интеракције,
2. Server – централна логика игре и оркестрација мрежне комуникације,
3. Shared – заједнички доменски модели и структура апликативних порука.

Оваквом организацијом обезбеђено је да кориснички интерфејс буде независан од критичне логике система. WPF клијент прикупља унос корисника и приказује стање игре, док сервер валидира све потезе, ажурира стање партије и прослеђује резултате свим учесницима.

Комуникација између модула реализована је преко TCP/UDP сокета, при чему се за размену користе типизирани поруке дефинисане у Shared слоју.

5.2 Структура података и заједнички модели

Како би се омогућила поуздана и брза размена кроз меморијске стримове мрежног интерфејса, сви кључни доменски објекти у дељеној библиотеци означени су атрибутом *Serializable*. То омогућава директну бинарну серијализацију у низ бајтова приликом транзита кроз TCP/UDP мрежне утичнице.

5.2.1 Класа Podmornica и структурирање флоте

Ентитет подморнице је моделован кроз класу Podmornica која садржи логику стања појединачне војне јединице. Димензије јединица су одређене преко енумерације TipPodmornice, чије вредности директно корелирају са физичком дужином (бројем поља) у матрици игре: Podmornica1x1 = 1 (4 подморнице), Podmornica2x1 = 2 (3 подморнице), Podmornica3x1 = 3 (2 подморнице) и Podmornica4x1 = 4 (1 подморница).

Унутар објекта се чувају колекције Pozicije (целобројни низови координата заузетих поља) и PogodjenePozicije (индекси поља које је противник успешно детонирао). Метода DodajPogodak (int pozicija) врши динамичку проверу — уколико се гађано поље налази у листи заузетих поља, оно се додаје у колекцију погодака. Истовремено се позива метода DaLiJePotopljena(), која пореди дужину листе јединствених погодака са укупним бројем позиција објекта, те у случају једнакости поставља поље Potopljena на логичку вредност true.

5.2.2 Класа Igrac и управљање меморијским стањем

Класа Igrac представља дигитални профил корисника на серверској страни и у себи држи стање свих меморијских структура неопходних за одвијање игре. Она садржи јединствени id, корисничко име (ime), две дводимензионалне матрице димензија 10x10 (matrica за позиције својих бродова и matricaGadjana за евиденцију напада на сопствену таблу), листу постављених објеката podmornice, као и бројаче за казнени систем (brojPromasaja и botTimeoutCount).

Један од најважнијих архитектонских детаља у овој класи је примена атрибута за искључивање серијализације [11] – [NonSerialized] изнад поља Socket који можемо видети на листингу 5.1.

```

[Serializable]
public class Igrac
{
    [NonSerialized]
    public Socket socket;
    public int id { get; }
    public string ime { get; set; }
    public int brojPromasaja { get; set; }
    public int botTimeoutCount { get; set; } = 0;
    public List<Podmornica> podmornice { get; set; } = new List<Podmornica>();
    public int[,] matrica { get; set; }
    public int[,] matricaGadjana { get; set; }
    public bool izgubio { get; set; }
    ...
}

```

Листинг 5.1. Имплементација класе *Igrac* са атрибутот *[NonSerialized]* изнад мрежне утичнице

Пошто мрежна утичница (*socket*) представља активни ресурс оперативног система везан за мрежни стек, она по својој природи не може бити серијализована у бајт-стрим. Постављањем овог атрибута, систем приликом мрежног слања целог играча потпуно игнорише и изузима сокет структуру, чиме се избегавају хардверски изузеци и омогућава неометан пренос логичког стања играча клијентима. Такође, класа пружа методе за двосмерну конверзију *PretvoriMatricuUString* и *PretvoriStringUMatricu*, што олакшава међупроцесну комуникацију.

5.2.3 Класа *Poruka* као апликативни мрежни протокол

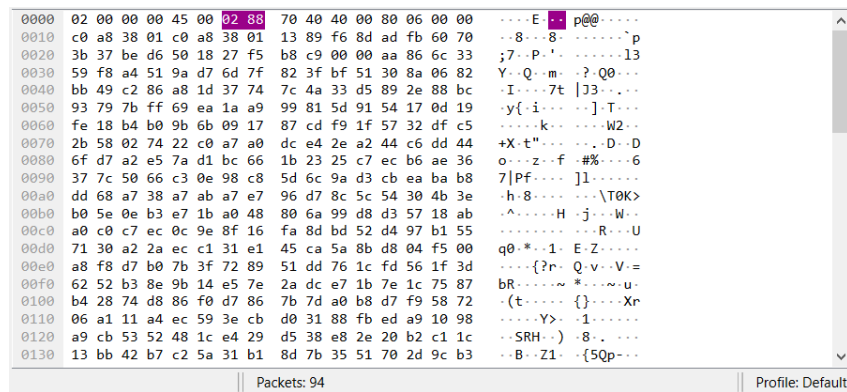
Сва комуникација на мрежном слоју обавља се стандардизовано кроз објекте класе *Poruka*. Она садржи референце на корисника који напада (*NaPotezu*), играча који је мета дејства (*Napadnut*), пратећи текстуални садржај (*poruka*) и кључно поље *tipPoruke* дефинисано кроз енумерацију *TipPoruke* (*PozicijeBrodova*, *Napad*, *Pogodak*, *Promasaj*, *TajmerOdbrojavanje*, *Botlgra*, *Kraj*, итд.).

Преко уграђених метода *Serializuj()* и статичке методе *DeserializujPoruku(byte[] bytes)*, класа коришћењем *BinaryFormatter*-а и *MemoryStream*-а аутоматски врши конверзије токова података на мрежним интерфејсима (Листинг 5.2).

У актуелној верзији система, процес серијализације је додатно проширен симетричним AES шифровањем пре слања поруке кроз мрежу. Након што се објект класе *Poruka* серијализује у бајт-низ, над добијеним подацима се примењује алгоритам AES у CBC режиму рада (*CipherMode.CBC*) са PKCS7 допуном (*PaddingMode.PKCS7*). На пријемној страни, примљени шифровани бајтови се најпре дешифрују истим кључем и иницијализационим вектором (IV), а затим десеријализују у оригинални објект *Poruka*.

На овај начин, комуникациони слој обезбеђује:

- 1) поверљивост порука током транспорта (садржај није читљив без кључа – Слика 5.1.),
- 2) заштиту тока игре од пасивног прислушкивања мреже,
- 3) очување постојећег апликативног протокола без промене логики виших слојева.



Слика 5.1. Шифрована порука „ухваћена“ у Wireshark програму

У имплементацији је шифровање интегрисано директно у методе `Serializuj()` и `DeserializujPoruku()`, што омогућава да остатак система (WPF клијент и серверска логика) користи исту класу `Poruka` без додатних измена у начину слања и пријема података.

```
public byte[] Serializuj()
{
    BinaryFormatter bf = new BinaryFormatter();
    using (MemoryStream temp = new MemoryStream())
    {
        bf.Serialize(temp, this);
        byte[] podaci = temp.ToArray();
        using (Aes aes = Aes.Create())
        {
            aes.Key = key;
            aes.IV = iv;
            aes.Mode = CipherMode.CBC;
            aes.Padding = PaddingMode.PKCS7;
            using (MemoryStream encrypted = new MemoryStream())
            {
                using (CryptoStream cs = new CryptoStream(
                    encrypted,
                    aes.CreateEncryptor(),
                    CryptoStreamMode.Write))
                {
                    cs.Write(podaci, 0, podaci.Length);
                    cs.FlushFinalBlock();
                }
                return encrypted.ToArray();
            }
        }
    }
}

public static Poruka DeserializujPoruku(byte[] bytes)
{
    using (Aes aes = Aes.Create())
    {
```

```

aes.Key = key;
aes.IV = iv;
aes.Mode = CipherMode.CBC;
aes.Padding = PaddingMode.PKCS7;
using (MemoryStream msEncrypted = new MemoryStream(bytes))
using (CryptoStream cs = new CryptoStream(
    msEncrypted,
    aes.CreateDecryptor(),
    CryptoStreamMode.Read))
using (MemoryStream msDecrypted = new MemoryStream())
{
    cs.CopyTo(msDecrypted);
    msDecrypted.Position = 0;
    BinaryFormatter bf = new BinaryFormatter();
    return (Poruka)bf.Deserialize(msDecrypted);
}
}
}

```

Листинг 5.2. Бинарна серијализација поруке и њено шифровање

5.3 Клијентска апликација (WPF специфичности)

Клијентска апликација је развијена коришћењем WPF (*Windows Presentation Foundation*) оквира, који омогућава јасно раздвајање логике приказа (XAML) и логике позадинског кода (C#).

- **Data Binding и MVVM елементи:** Уместо директне манипулације контролама, апликација користи својства везивања (data binding) за ажурирање статуса табле у реалном времену. Када сервер пошаље информацију о поготку, MainWindow кроз Dispatcher нит ажурира ObservableCollection колекцију, што аутоматски освежава графички приказ поља на екрану без потребе за ручним поновним исцртавањем (redrawing).
- **Визуелни повратни списак (Visual Feedback):** Уведене су анимације и промене боја поља (зелено за погодак, црвено за промашај) како би се играчу омогућила интуитивна интеракција. Коришћени су Trigger-и у XAML-у који реагују на промену стања Podmornica објекта.

5.4 Логика иницијализације и регистрације играча

Процес повезивања и оркестрације корисника на мрежи имплементиран је двостепено, како би се максимално искористиле предности оба транспортна протокола:

- **UDP Фаза (Мрежно откривање и пријава):** Након што корисник покрене конзолну апликацију и унесе своје име, клијентски сокет шаље почетни UDP датаграм са унесеним стрингом на дефинисани порт сервера. На серверској страни налази се неблокирајући UDP сокет који континуирано ослушкује мрежни сегмент. По пријему датаграма, сервер издваја IP адресу и порт пошиљаоца, инстанцира објекат Klijent и смешта га у глобалну листу Klijenti. Ово омогућава брзо прикупљање захтева заинтересованих играча без трошења ресурса на успостављање сталних конекција.

- **TCP Фаза** (Покретање игре и трајне везе): Када сервер идентификује да је попуњен очекивани капацитет играча дефинисан променљивом `MaxBrojIgraca`, покреће се главна сесија игре. Сервер шаље повратни сигнал клијентима који тада аутоматски иницирају метод `Connect()` ка главном TCP сокету сервера (`serverSocket`). Сервер извршава метод `Accept()` и везује наменски TCP сокет за сваког појединачног играча. Мрежни ток прелази на потпуно конзистентан и сигуран TCP режим преноса серијализованих објеката класе `Poruka`.

5.5 Валидација и правила постављања подморница

Након успешне TCP регистрације, корисници прелазе на локално постављање подморница унутар својих матрица. Логика система стриктно спроводи правила званичне игре кроз неколико нивоа валидације пре него што дозволи слање табле серверу:

- **Матрична провера граница:** Клијент за сваку подморницу бира поље на табли (координате у матрици), уз избор оријентације (`Horizontalna/Vertikalna`). Систем итеративно додаје поља у зависности од дужине типа подморнице (1 до 4 поља). Ако израчунати индекси прелазе `VelicinaTable`, апликација прекида петљу, одбија постављање и упозорава корисника.
- **Спречавање додиривања подморница:** Ниједна подморница не може бити постављена у околним пољима већ постојеће подморнице.
- **Спречавање преклапања:** Ниједна подморница не сме користити поље на ком би дошло до преклапања са другом подморницом, што можемо видети на Листингу 5.3.

```
private List<int> GenerisiPozicijePodmornice(int x, int y, int duzina, bool horizontalna,
int velTable, HashSet<int> zauzetePozicije)
{
    List<int> potencijalnePozicije = new List<int>();
    for (int i = 0; i < duzina; i++)
    {
        int nx = horizontalna ? x : x + i;
        int ny = horizontalna ? y + i : y;
        if (nx < 1 || nx > velTable || ny < 1 || ny > velTable)
            return null;
        int poz = (nx - 1) * velTable + ny;
        if (zauzetePozicije.Contains(poz))
            return null;
        potencijalnePozicije.Add(poz);
    }
    foreach (int p in potencijalnePozicije)
    {
        int px = ((p - 1) / velTable) + 1;
        int py = ((p - 1) % velTable) + 1;

        for (int dx = -1; dx <= 1; dx++)
        {
            for (int dy = -1; dy <= 1; dy++)
            {
```

```

        int susedX = px + dx;
        int susedY = py + dy;
        if (susedX >= 1 && susedX <= velTable && susedY >= 1 && susedY <=
velTable)
        {
            int susedPoz = (susedX - 1) * velTable + susedY;
            if (zauzetePozicije.Contains(susedPoz) &&
!potencijalnePozicije.Contains(susedPoz))
            {
                return null;
            }
        }
    }
}
return potencijalnePozicije;
}

```

Листинг 5.3. Валидација постављања подморница при иницијализацији табле

- Потврда флоте: Када се успешно валидира комплетна флота од 10 подморница, клијент стање своје табле шаље серверу кроз тип поруке `TipPoruke.PozicijeBrodova`. Сервер прихвата ове податке и аутоматски позива класу `Logger.LogPodmornice(igrac, podmornice)`, перзистирајући почетни распоред снага у текстуални лог фајл за ту партију.

5.6 Логика играња потеза

Срж практичног решења чини динамички, потезни систем који омогућава кружно смењивање играча на потезу, слање напада и моменталну дистрибуцију резултата гађања свим учесницима у партији.

Игра се одвија по принципу цикличних потеза. Унутар серверске апликације одређује се корисник који је тренутно на потезу (`NaPotezu`) и шаље сигнал да је систем спреман да прихвати његове координате напада. На клијентској страни, кориснику се отвара могућност играња: бира поље на противничкој табли које жели да гађа, при чему може изабрати да тип потеза буде и `SuperPotez` (уколико није искоришћен).

Након уноса, клијент пакује ове информације у објекат класе `Poruka` и шаље их назад. Сервер преузима пакет, идентификује ко је мета напада (`Napadnut`) и приступа матричној провери:

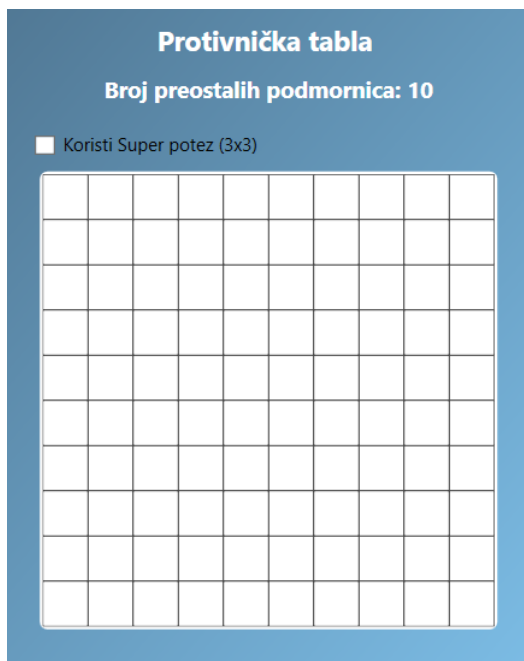
- Промашај: Уколико је циљано поље у матрици нападнутог играча имало вредност 0 (нема подморнице на том месту), сервер ту вредност мења у казнени статус, инкрементира бројач узастопних промашаја нападачу, логује догађај преко `Logger.LogPotez` и пребацује потез на следећег активног играча.
- Погодак: Уколико поље садржи део подморнице (вредност већа од 0), сервер прослеђује индекс поља конкретном објекту `Podmornica` кроз метод `DodajPogodak()`. Ако брод није потпуно уништен, сервер враћа поруку типа `TipPoruke.Pogodak`, а играч који је успешно детонирао поље задржава право на потез и може поново да гађа.

- Потапање: Ако је унети погодак уједно и последње поље те подморнице, флег Potopljena се поставља на true. Сервер тада свим играчима шаље глобално обавештење да је одређени тип подморнице званично уништен, што се јасно ажурира на графичком приказу табли.

5.7 Специјални супер потез (9 поља одједном)

Као напредна функционалност пројекта, имплементирана је опција "Супер-потеза" која драстично мења тактички ток игре. Сваки играч поседује могућност да само једном у току партије изврши масивни удар који обухвата површину од 9 поља одједном (матрица 3x3 око циљане тачке).

Клијентски интерфејс нуди ову опцију приликом сваког редовног потеза (Слика 5.2.) све док се вредност променљиве superPotezIskoriscen не промени из false у true.



Слика 5.2. Могућност избора супер потеза при игрању потеза

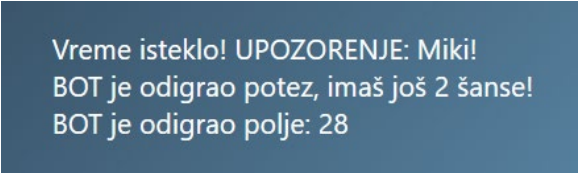
Када играч одабере ову опцију и изабере централну координату напада, клијент шаље поруку са посебном ознаком у текстуалном пољу. Сервер након пријема поруке израчунава координате за свих девет поља. Пре извршења, алгоритам на серверу проверава границе табле за сваку од девет изведених локација — уколико су нека околна поља ван опсега матрице (нпр. када се гађа сама ивица табле), та поља се једноставно игноришу, док се над валидним пољима покреће стандардни поступак детонације. Сваки погодак или промашај унутар ове 3x3 зоне се појединачно рачуна и ажурира у матрици противника. Овај догађај се у целости региструје преко методе Logger.LogPotez().

5.8 Временско ограничење, казнени поени и бот преузимање контроле

Како би се спречило намерно или случајно блокирање тока партије услед неактивности клијената, у систем је интегрисан динамички тајмер систем преко класе System.Timers.Timer [\[12\]](#). Овај тајмер ради аутономно у засебној нити оперативног система и одбројава време предвиђено за потез (дефинисано променљивом sekundeCekanjaNaUnos).

Алгоритам реакције на истек времена:

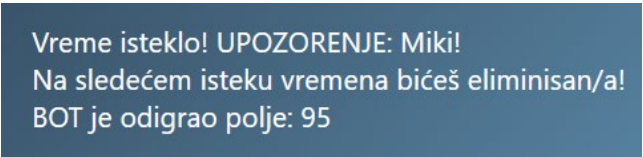
- Први прекршај тајмера: Уколико вредност тајмера достигне нулу пре него што је активирани клијент послао потез, сервер зауставља одбројавање и инкрементира вредност `botTimeoutCount` у профилу тог играча. Сервер тада преузима извршну контролу и преусмерава је на класу `BotHelper` (имплементира бот играча). Покреће се метода `BotHelper.IzaberiRandomIgraca()` који насумично одабира активну мету, као и метода `BotHelper.IzaberiRandomPolje()`, која генерише случајну координату напада. Овај генерисани потез се аутоматски извршава на серверу. Клијенту који је закаснио шаље се пакет са типом `TipPoruke.BotIgra` као обавештење да је његов потез аутоматизован (Слика 5.3.).



Vreme isteklo! UPOZORENJE: Miki!
BOT je odigrao potez, imaš još 2 šanse!
BOT je odigrao polje: 28

Слика 5.3. Први прекршај тајмера

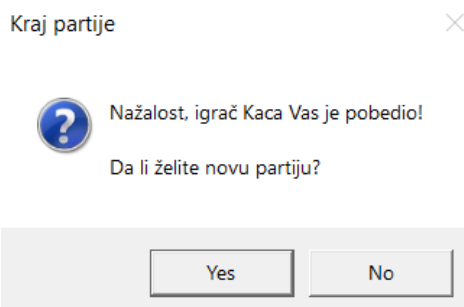
- Други узастопни прекршај (Упозорење): Приликом следећег покретања потеза истог играча, тајмер се ресетује. Ако време поново истекне, клијенту се одмах прослеђује серијализована порука са строгим упозорењем да ће наредно кашњење резултирати моменталном дисквалификацијом. Затим, сервер поново одигра аутоматски потез (Слика 5.4.).



Vreme isteklo! UPOZORENJE: Miki!
Na sledećem isteku vremena bićeš eliminisan/a!
BOT je odigrao polje: 95

Слика 5.4. Други прекршај тајмера - упозорење

- Трећи прекршај (Елиминација из партије): Ако корисник и трећи пут узастопно прекорачи временско ограничење, његово стање се аутоматски мења постављањем променљиве `izgubio = true`, а његов TCP сокет се искључује из активне листе за мултиплексирање. Сервер шаље поруку типа `TipPoruke.Izgubio` елиминисаном играчу, док остали корисници добијају обавештење да је противник дисквалификован услед неактивности и настављају игру без њега (Слика 5.5.). Сваки корак овог процеса перзистентно уписује метода `Logger.LogIgrac()`.



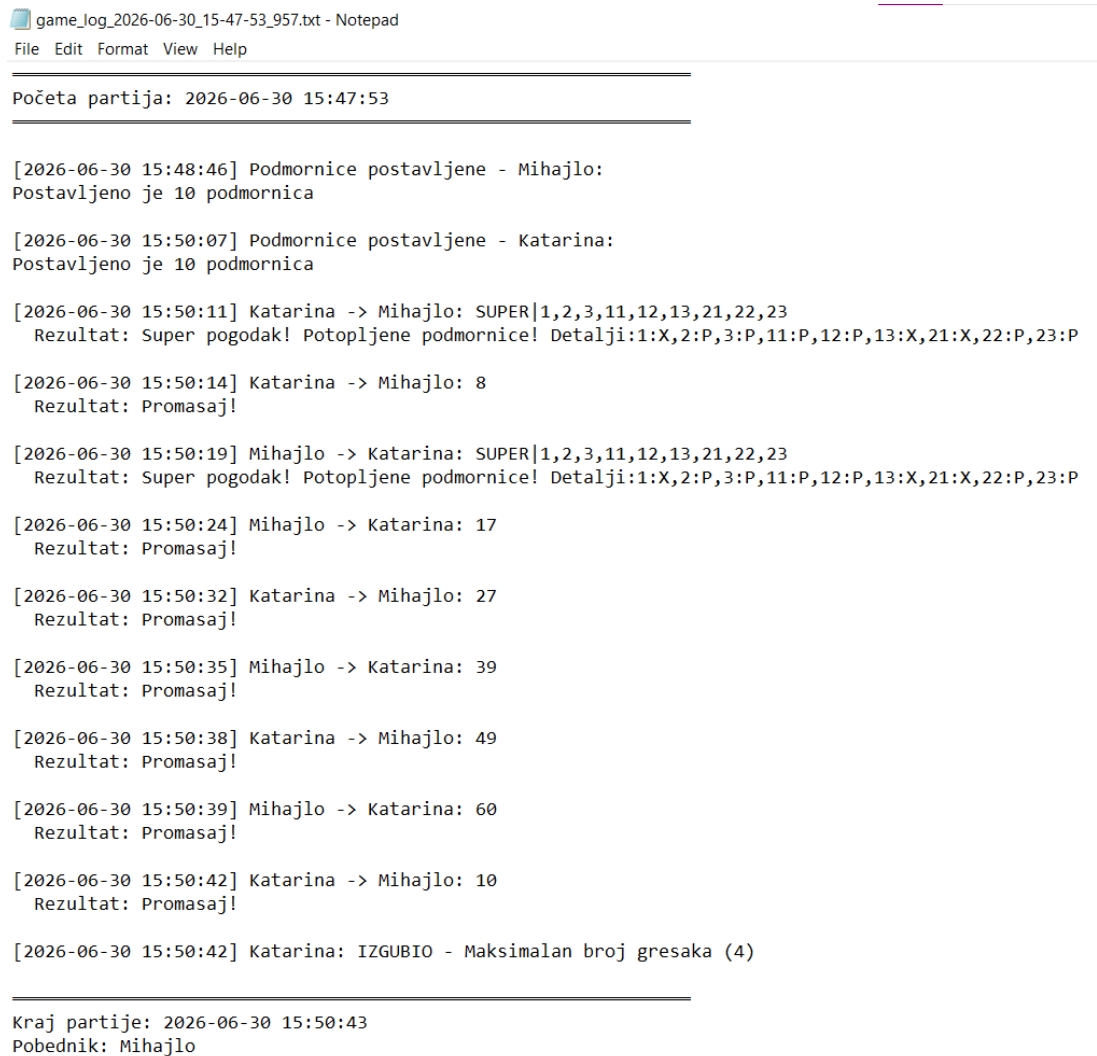
Слика 5.5. Трећи прекршај тајмера - елиминација

Поред временских казни, систем строго прати и казне за узастопне промашаје: уколико играч оствари вредност `brojPromasaja == MaxUzastopnihGresaka` кроз неуспешна редовна гађања, систем га такође аутоматски проглашава пораженим, чиме се игра

динамички убрзава и чисти. Партија се завршава када на табли остане само један играч чији флег izgubio има вредност false, након чега се свима шаље пакет TipPoruke.Kraj са финалним резултатима.

5.9 Бележење тока игре у документ

При свакој кључној извршеној операцији позива се Logger који те акције бележи у посебан документ за сваку партију (Слика 5.6.)



```
game_log_2026-06-30_15-47-53_957.txt - Notepad
File Edit Format View Help

Početa partija: 2026-06-30 15:47:53

[2026-06-30 15:48:46] Podmornice postavljene - Mihajlo:
Postavljeno je 10 podmornica

[2026-06-30 15:50:07] Podmornice postavljene - Katarina:
Postavljeno je 10 podmornica

[2026-06-30 15:50:11] Katarina -> Mihajlo: SUPER|1,2,3,11,12,13,21,22,23
Rezultat: Super pogodak! Potopljene podmornice! Detalji:1:X,2:P,3:P,11:P,12:P,13:X,21:X,22:P,23:P

[2026-06-30 15:50:14] Katarina -> Mihajlo: 8
Rezultat: Promasaj!

[2026-06-30 15:50:19] Mihajlo -> Katarina: SUPER|1,2,3,11,12,13,21,22,23
Rezultat: Super pogodak! Potopljene podmornice! Detalji:1:X,2:P,3:P,11:P,12:P,13:X,21:X,22:P,23:P

[2026-06-30 15:50:24] Mihajlo -> Katarina: 17
Rezultat: Promasaj!

[2026-06-30 15:50:32] Katarina -> Mihajlo: 27
Rezultat: Promasaj!

[2026-06-30 15:50:35] Mihajlo -> Katarina: 39
Rezultat: Promasaj!

[2026-06-30 15:50:38] Katarina -> Mihajlo: 49
Rezultat: Promasaj!

[2026-06-30 15:50:39] Mihajlo -> Katarina: 60
Rezultat: Promasaj!

[2026-06-30 15:50:42] Katarina -> Mihajlo: 10
Rezultat: Promasaj!

[2026-06-30 15:50:42] Katarina: IZGUBIO - Maksimalan broj gresaka (4)

Kraj partije: 2026-06-30 15:50:43
Pobednik: Mihajlo
```

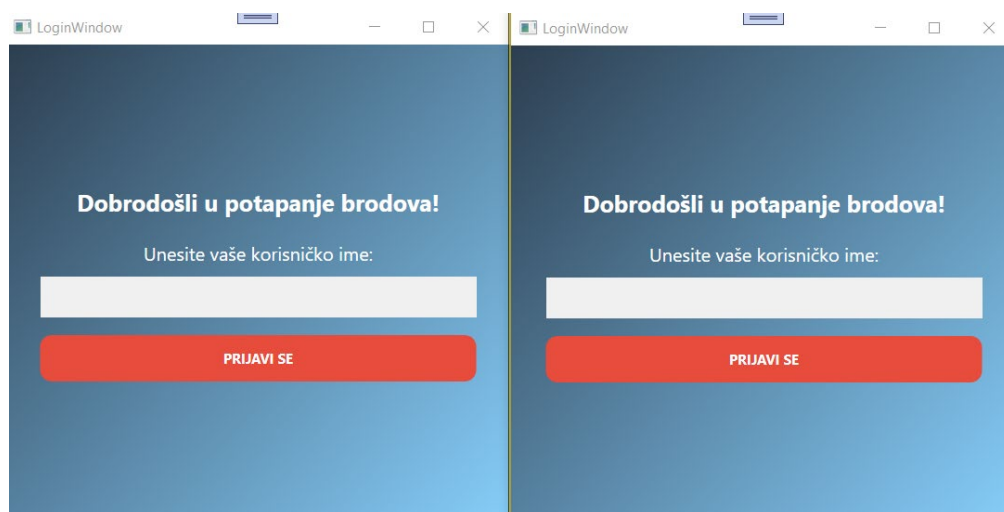
Слика 5.6. Пример једне партије забележене у документ

6. Верификација квалитета решења

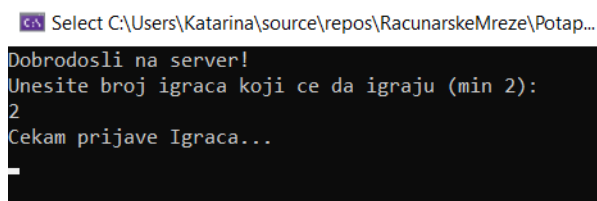
6.1 Покретање и повезивање

Након успешног покретања серверске апликације, која отвара портове за комуникацију, корисник покреће ClientWPF апликацију. Приликом првог покретања, кориснику се приказује LoginWindow (Слика 6.1.).

- **Унос параметара:** Корисник је у обавези да унесе јединствено корисничко име.
- **Иницијализација:** Кликот на дугме за пријаву, апликација шаље UDP пакет серверу ради регистрације играча у систему.
- **Потврда повезивања:** Уколико су подаци исправни и сервер је доступан, систем враћа повратну информацију SPREMAN, након чега се отвара главни прозор игре.

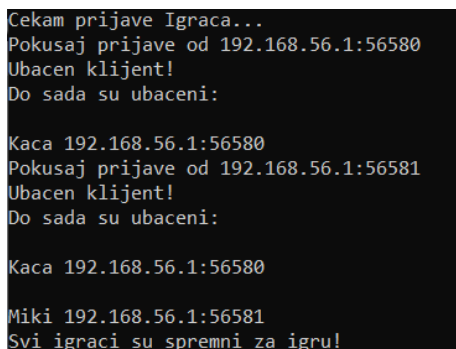


Слика 6.1. Интерфејс за пријаву корисника

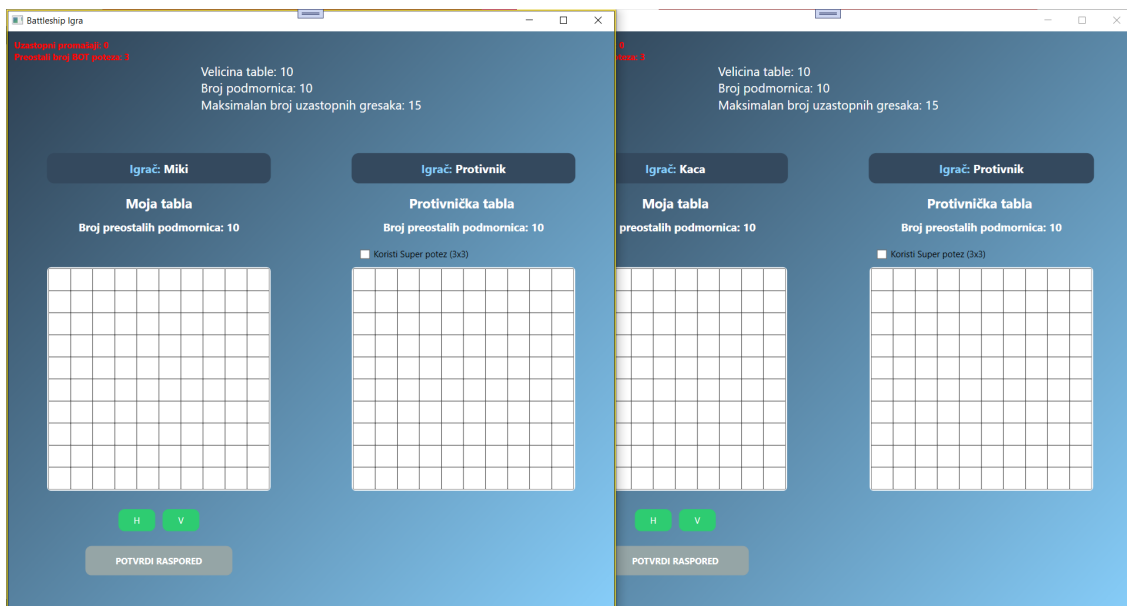


Слика 6.2. Почетно стање сервера

Успешност повезивања се верификује ажурирањем интерфејса у главном прозору клијентске апликације (Слика 6.4.) и приказом повезаних корисника на серверу (Слика 6.3.). Сва комуникација након почетне UDP пријаве прелази на TCP протокол, чиме се обезбеђује поуздан пренос података о потезима и стању табле током целе партије.



Слика 6.3. Ажурирани сервер са приказом повезаних корисника



Слика 6.4. Ажурирани интерфејс након успешног повезивања клијената на сервер

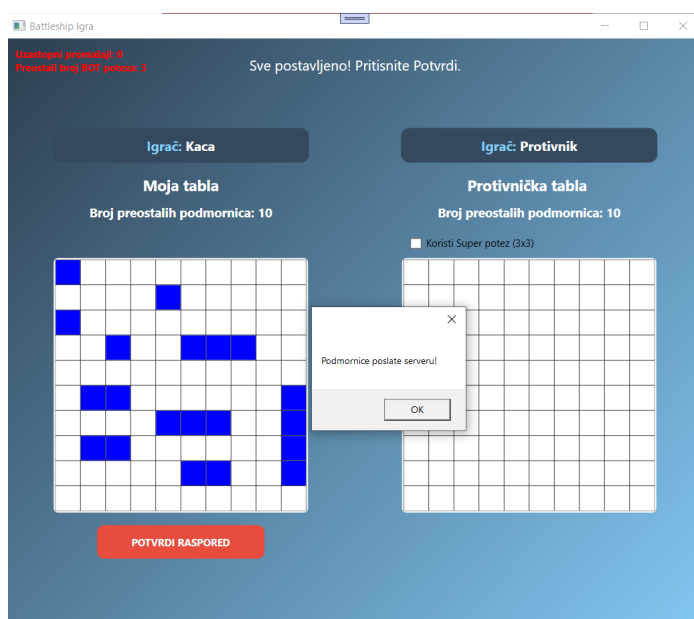
6.2 Постављање подморница и припрема игре

Након што је играч успешно повезан са сервером, прелази се на фазу припреме табле. Ова фаза је кључна јер дефинише почетну позицију свих подморница и омогућава верификацију правила пре почетка борбе.

Процес распоређивања: Главни прозор апликације садржи `MojaTablaGrid`, на којој играч поставља своје подморнице. Корисник има могућност ручног постављања 10 подморница (4x1, 3x2, 2x3, 1x4).

Интерактивност: Приликом клика на жељена поља на табли, апликација визуелно обележава позиције подморница.

Валидација: Систем садржи уграђену логику која спречава неодговарајуће потезе, попут преклапања подморница, додиривања подморница или постављања изван граница саме табле.



Слика 6.5. Приказ табле након успешно распоређених подморница

```

Poruka poslata klijentu: Tip poruke: Obavestenje
Poruka poslata klijentu: Tip poruke: Obavestenje
Podmornice od Kaca: 1,H,1;1,H,21;1,H,15;1,H,33;2,H,52,53;2,H,72,73;2,H,86,87;3,H,65,66,67;3,H,36,37,38;4,V,60,70,80,90
Podmornice od Kaca prihvacene (10/10)

```

Слика 6.6. Приказ прихваћених подморница на серверу

Када су све подморнице постављене према правилима, играч је спреман за почетак партије. Ова фаза осигурава да су сви подаци о позицијама исправно сачувани у локалном стању клијента пре него што се пошаљу серверу ради покретања саме борбе.

6.3 Ток игре и комуникација

Након што су обе стране поставиле своје подморнице, отпочиње фаза борбе. Ово поглавље описује како корисник изводи напад и како се врши размена података између клијента и сервера.

Извођење напада: Играч врши напад кликом на поље унутар ProtivnickaTablaGrid. Апликација тада шаље TCP поруку серверу која садржи координате изабраног поља. Уколико је активиран CheckBox „Супер потез“, апликација шаље специфичан захтев за гађање 3x3 области, чиме се повећава шанса за погодак (Слика 6.7.)

```

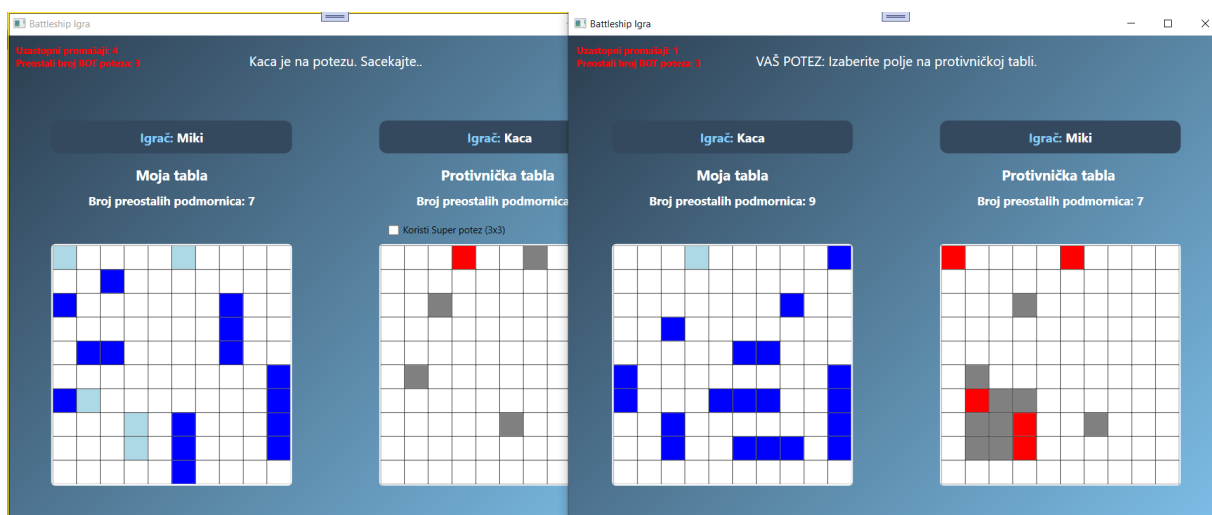
Poruka poslata igracu Miki: Kaca je na potezu. Sacekajte.. Tip poruke: Obavestenje
Poruka poslata igracu Kaca: Izaberi koga zelis da napadnes
->Miki Tip poruke: Napad
Primljen odgovor od Kaca: Miki
Primljen odgovor od Kaca: SUPER|62,63,64,72,73,74,82,83,84

```

Слика 6.7. Обрада захтева за гађање супер потеза

Визуелна повратна информација: Систем ажурира стање MojaTablaGrid (таблу играча који је нападнут тако што се потопљене подморнице означавају светло плавом бојом) уколико се догодио погодак и ProtivnickaTablaGrid (противничку таблу коју играч гађа) на основу одговора који стиже са сервера:

- Погодак: Поља на противничкој табли се означавају црвеном бојом када је подморница успешно погођена.
- Промашај: Поља на противничкој табли се означавају сивом бојом када на гађаној локацији нема подморнице.



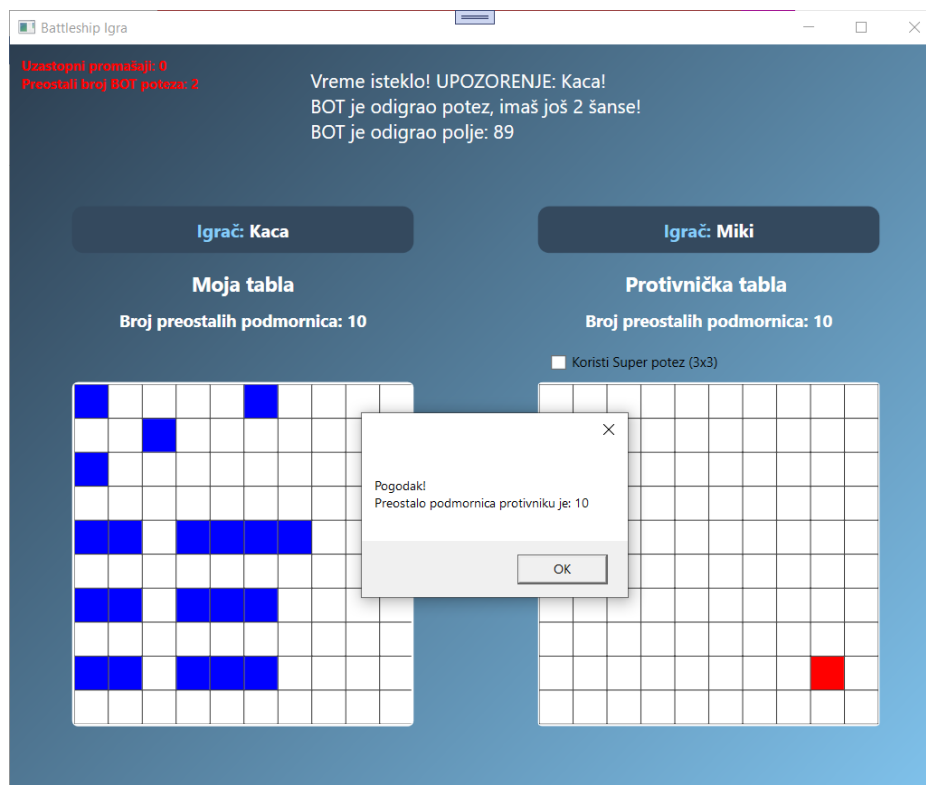
Слика 6.8. Приказ табле током активне размене напада.

Овај механизам омогућава синхронизацију у реалном времену између два играча. Кроз статус поље у апликацији (StatusUnosa), корисник увек има увид у то да ли је на реду и да ли је његов претходни напад био успешан.

6.4 Механизам аутоматизације (играчки бот)

Последња фаза тестирања обухвата верификацију понашања система у условима када играч није активан, као и поступак завршетка партије.

Аутоматизација (бот играч): Систем је дизајниран тако да обезбеди континуитет игре чак и у случају спорог одговора играча. Сервер прати време које је играч утрошио на потез. Уколико време пређе дефинисану границу (`sekundeCekanjaNaUnos`), сервер аутоматски преузима контролу над потезом.



Слика 6.9. Одиграни бот потез

Активира се уграђени бот који уместо играча врши напад на противничку таблу, о чему играч добија обавештење кроз апликацију.

Корисник има право искоришћења бота 2 пута, а уколико се искористи и 3. пут, играч је елиминисан и противник је победио. (Слика 6.10.)

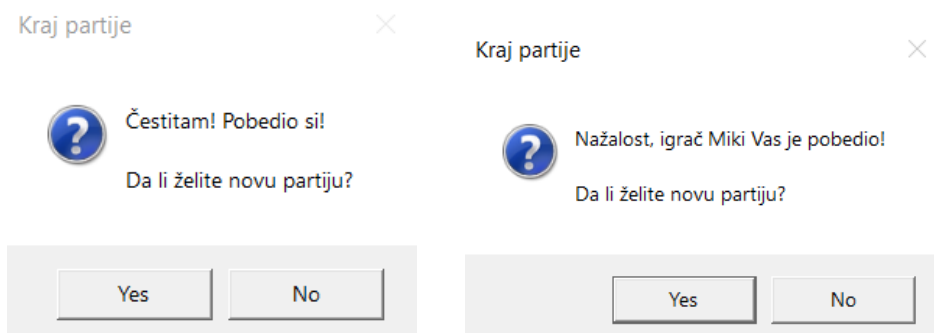
```
TAJMER ISTEKAO
Poruka poslata klijentu: Kaca
Eliminisan igrac Kaca
Poruka poslata igracu Miki: Kaca je eliminisan/a zbog isteka vremena!
Poruka poslata igracu Kaca: Kraj partije! Igrac Miki je pobedio! Tip poruke: GlasanjeNova
Poruka poslata igracu Miki: Kraj partije! Igrac Miki je pobedio! Tip poruke: GlasanjeNova
```

Слика 6.10. Елиминација корисника након 3. бот потеза

6.5 Крај игре и нова игра

Након што један од играча потопи све подморнице противника, апликација покреће завршну процедуру:

- Обавештење: Кориснику се приказује `MessageBox` са статусом победе или пораза.
- Гласање: Након завршене партије, сервер иницира процес гласања за покретање нове рунде, чиме се играчима омогућава наставак играња без поновног покретања апликације.

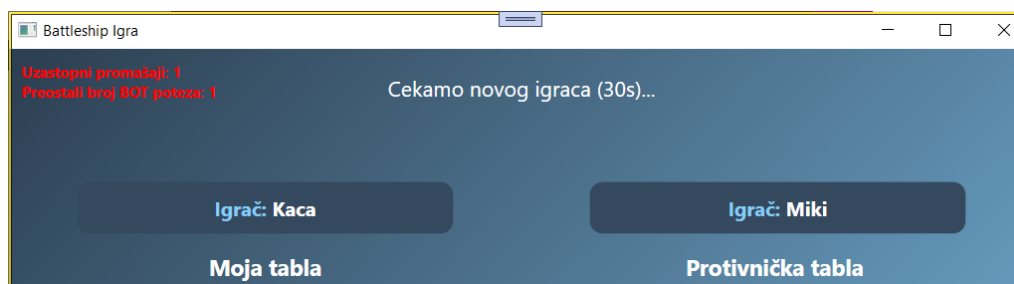


Слика 6.11. Обавештења о исходу игре и гласање за нову игру

Уколико оба играча гласају за покретање нове игре, партија се ресетује и започињу поновно постављање подморница.

Уколико оба играча нису за почетак нове игре, апликација се гаси.

Уколико један играч гласа НЕ за покретање нове игре, а други гласа ДА, играч који је гласао ДА остаје у стању чекања 30 секунди на пријаву неког другог (новог) играча са којим би одиграо партију. (Слика 6.12.)



Слика 6.12. Чекање на пријаву новог играча

7. Закључак

У оквиру овог пројекта успешно је реализован неблокирајући вишекориснички софтверски систем кроз имплементацију мрежне игре „Потапање подморница“. Кроз практичну примену на .NET платформи и програмском језику C#, демонстрирано је како се правилно пројектује архитектура апликација која захтева симултану интеракцију корисника у реалном времену.

Развојем овог система успешно су превазиђени традиционални проблеми синхроности мрежних утичница. Применом хибридног приступа постигнут је оптималан баланс између брзине и поузданости мрежног саобраћаја. Посебан инжењерски значај у раду има имплементација механизма мултиплексирања помоћу функције `Socket.Select`. Овај приступ је омогућио да сервер унутар једне јединствене извршне нити паралелно опслужује већи број клијената, чиме је хардверско оптерећење оперативног система сведено на минимум, а систем заштићен од унутрашњих блокирања (deadlock ситуација).

Поред основне логике игре, софтверско решење је додатно унапређено увођењем строгих валидационих правила постављања подморница, шифровањем порука које се размењују, подршком "Супер-позеа" великих димензија, као и напредног казненог система вођеног асинхроним тајмером. Интеграција аутоматизованог бота преко `BotHelper` класе и система за перзистенцију логова преко синхронизоване `Logger` класе осигурала је континуитет игре чак и у случајевима неактивности или дисквалификације појединих корисника, који задржавају улогу пасивних посматрача мрежних дешавања.

7.1 Унапређење постојећег софтверског решења

Иако реализовани систем нуди потпуну функционалност и високу стабилност, свако софтверско решење је подложно даљем унапређењу. У циљу подизања корисничког искуства и перформанси на виши ниво, могу се размотрити следеће измене:

- Графичко обогаћивање: Имплементација анимираних ефеката приликом поготка или промашаја подморница, као и увођење звучних записа који би додатно допринели динамици игре.
- Имплементација базе података: Тренутна перзистенција логова се може проширити на пуну интеграцију SQL базе података ради вођења историје мечева, рангирања играча (Leaderboard) и праћења личне статистике.
- Унапређење интерфејса: Редизајн корисничког интерфејса кроз употребу WPF стилова и тема (нпр. "Dark mode"), као и увођење могућности подешавања распореда елемената према жељама корисника.
- Интеграција Chat система: Додавање модула за текстуалну комуникацију у реалном времену између играча током партије, што би додатно побољшало интерактивност.
- Оптимизација бота: Додавање напредније вештачке интелигенције за бота (нпр. алгоритам заснован на вероватноћи), како би пружао изазовније искуство играчима у фази аутоматизације.

Литература

- [1] W. Stallings, Data and Computer Communications, 10th ed. Boston, MA: Pearson, 2014.
- [2] B. A. Forouzan, Data Communications and Networking, 5th ed. New York, NY: McGraw-Hill, 2013.
- [3] Microsoft Learn – AES: Keeping Your Data Secure with Advanced Encryption Standard [Online].
Доступно на: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2003/november/aes-keeping-your-data-secure-with-advance-encryption-standard>
(приступљено: август 2026).
- [4] J. Skeet, C# in Depth, 4th ed. Shelter Island, NY: Manning Publications, 2019.
- [5] Microsoft Learn, "Socket.Select Method (System.Net.Sockets)," microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.socket.select>
(приступљено: август 2026).
- [6] Microsoft Learn, "Binary Serialization in .NET," microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/binary-serialization>
(приступљено: август 2026).
- [7] Microsoft Learn, "SerializableAttribute Class (System)," microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.serializableattribute>
(приступљено: август 2026).
- [8] Microsoft Learn, "Windows Presentation Foundation," microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/>
(приступљено: јул 2026).
- [9] Microsoft Learn, "UdpClient Class (System.Net.Sockets)," [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.udpclient>
(приступљено: јул 2026).
- [10] Microsoft Learn, "TcpClient Class (System.Net.Sockets)," [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient>
(приступљено: јул 2026).
- [11] Microsoft Learn, "NonSerializedAttribute Class (System)," learn.microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.nonserializedattribute>
(приступљено: јул 2026).
- [12] Microsoft Learn, "Timer Class (System.Timers)," learn.microsoft.com. [Online].
Доступно на: <https://learn.microsoft.com/en-us/dotnet/api/system.timers.timer>
(приступљено: јул 2026).

Подаци о кандидату



Катарина Вељковић рођена је 2004. године у Смедереву. Основну школу „Бранко Радичевић“ у Смедереву завршила је као носилац Вукове дипломе. Након завршене основне школе уписала је Техничку школу у Смедереву, образовни профил електротехничар информационих технологија, коју је завршила са одличним успехом.

Поред редовног школовања, завршила је Нижу музичку школу „Коста Манојловић“ у Смедереву са одличним успехом. Активно се бави јапанским џиу-џицуом, носилац је црног појаса 1. Дан и остварила је запажене резултате на домаћим и међународним такмичењима.

Школске 2022/2023. године уписала је Факултет техничких наука Универзитета у Новом Саду, студијски програм Примењено софтверско инжењерство.

Испунила је све обавезе и положила све испите предвиђене студијским програмом са просечном оценом 9,29.